Statistical and Machine Learning

Exemplar and kernel methods

Katarzyna Reluga

Exemplar and kernel methods

- Exemplar methods
  - k-nearest neighbor (kNN)
  - kernel density estimation (KDE)
- Kernel methods
  - Mercer Kernels
  - Gaussian process (GP)
  - Support Vector Machine (SVM)

Exemplar methods vs kernel methods

Overview !

- **Exemplar methods** predict using stored training examples.
- They are often **local**: nearby observations matter most.
- Examples: k-**NN**, **kernel density estimation**.

- **Kernel methods** use a kernel $K(\mathbf{x}, \mathbf{z})$ as an implicit **similarity / inner-product function**.
- They often rely on the **kernel trick**. → later !
- Examples: **SVM**, **kernel ridge regression**, **Gaussian processes**.

Key difference

Exemplar methods use stored examples directly, often with local distance-based weighting.

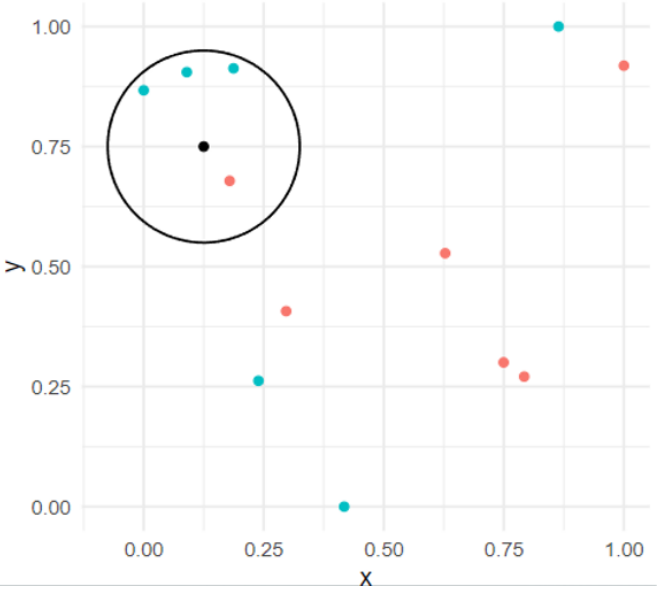
Kernel methods use positive semidefinite kernels as implicit inner products in feature space.

Exemplar and kernel methods

- Exemplar methods
  - **k-nearest neighbor (kNN)**
  - kernel density estimation (KDE)
- Kernel methods
  - Mercer Kernels
  - Gaussian process (GP)
  - Support Vector Machine (SVM)

k-nearest neighbor – basic idea

- Points that are similar in the predictor space are expected to have similar responses.
- Given a new point (black), which class do you think it is more likely to belong to?



k-nearest neighbor – classification rule

- Consider a classification setting.
- The k-NN model estimates the probability that an observation with features $\mathbf{x}$ belongs to class l based on the labels of the k nearest neighbors of $\mathbf{x}$, denoted by N_x^k.

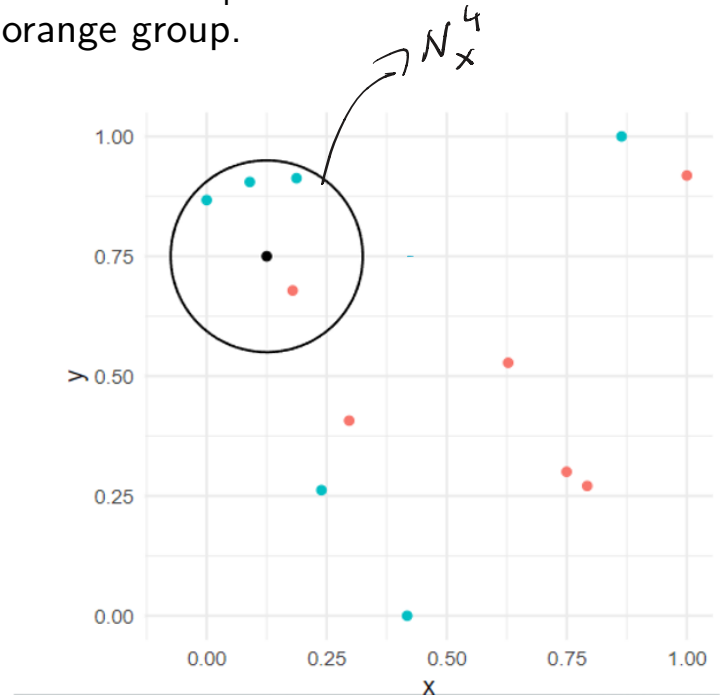
$$\hat{P}(Y = l \mid \mathbf{x}) = \frac{1}{k} \sum_{i \in N_x^k} 1_{\{y_i = l\}}$$

- The term $\sum_{i \in N_x^k} 1_{\{y_i = l\}}$ counts how many of the k nearest neighbors belong to class l.
- Dividing by k gives the estimated probability.

k-nearest neighbor – example

Suppose $k = 4$ is chosen. At the candidate black point. The four nearest neighbors are inspected.

There is a probability of $\frac{3}{4}$ of being in the green group and $\frac{1}{4}$ for being in the orange group.



k-nearest neighbor – regression

- Consider a regression setting.
- The k-NN model predicts the response value for an observation with features $\mathbf{x}$ based on the response values of the k nearest neighbors of $\mathbf{x}$, denoted by N_x^k.

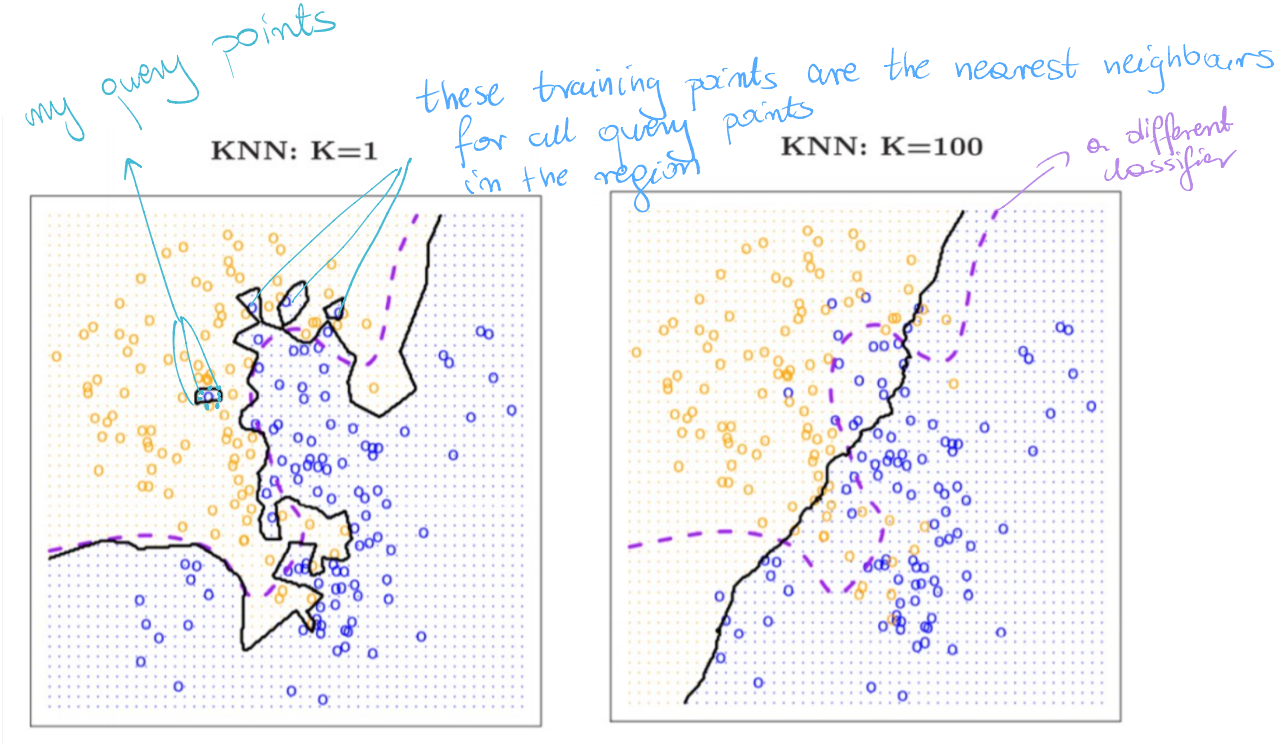
$$\hat{f}(\mathbf{x}) = \hat{\mathbb{E}}(Y \mid \mathbf{x}) = \frac{1}{k} \sum_{i \in N_x^k} y_i$$

summing y_i in the neighbourhood N_x^k

- The term $\sum_{i \in N_x^k} y_i$ adds the response values of the k nearest neighbors.
- Dividing by k gives the average response among the nearest neighbors, which is the k-NN regression prediction.

average

k-nearest neighbor – differnt choices of k



How do we choose N_x^k?

- $N_{\mathbf{x}}^k$ depends on the choice of distance metric.
- Euclidean distance:

$$L_2(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{l=1}^{p} |x_{il} - x_{jl}|^2 \right)^{1/2}$$

- More generally, the L_q distance is

$$L_q(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{l=1}^{q} |x_{il} - x_{jl}|^q \right)^{1/q}$$

- Manhattan distance is the special case $q = 1$:

$$L_1(\mathbf{x}_i, \mathbf{x}_j) = \sum_{l=1}^{q} |x_{il} - x_{jl}|$$

- The set $N_{\mathbf{x}}^k$ contains the k training points closest to $\mathbf{x}$ under the chosen distance.

kNN vs. LDA vs. Logistic Regression

- k-NN takes a completely different approach.
- k-NN is fully nonparametric: it makes no assumptions about the shape of the decision boundary.
- Advantage of k-NN: it can outperform both LDA and logistic regression when the decision boundary is highly nonlinear.
- Disadvantage of k-NN: it does not indicate which predictors are important, since there is no table of coefficients.

no p-value, correlation — only the closeness

- scales poorly with large n and p

(storing and searching through all points!)

Exemplar and kernel methods

- Exemplar methods
  - k-nearest neighbor (kNN)
  - **kernel density estimation (KDE)**
- Kernel methods
  - Mercer Kernels
  - Gaussian process (GP)
  - Support Vector Machine (SVM)

Density estimation

- In exploratory data analysis, density estimation can be used:
  - to assess multimodality, skewness, tail behavior, etc.,
  - in decision-making, classification, and the summarization of Bayesian posteriors,
  - as a useful visualization tool, providing a simple summary of a distribution.
- Suppose random variables $\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_n$ have been observed and are assumed to be sampled independently from a distribution with density f.
- Goal: to estimate the density function f.

Density estimation

- Parametric method: assume a parametric model, e.g., assume that your data follows a normal distribution:

$$\mathbf{X}_1, \ldots, \mathbf{X}_n \sim f_\theta(\mathbf{x}), i.i.d.$$
$$\boldsymbol{\theta} = (\boldsymbol{\mu}, \boldsymbol{\Sigma}) \text{ and } \mathbf{X} \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

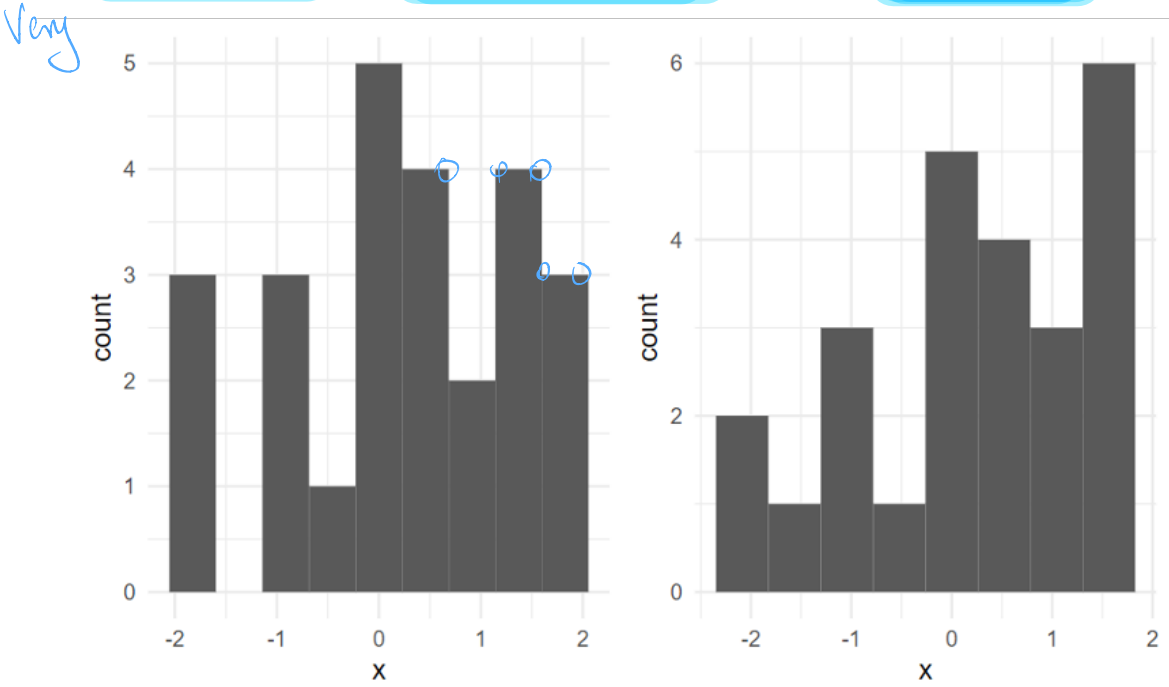
Model class indexed through $\theta \in \Theta$

- We can use MLE to estimate parameters
- What if the data do not follow parametric assumption?
  - Histogram!

> visualization (not the inference)

Histogram

- Crude density estimate (kinky edges of bins)
- Offers a very simple visualization
- Sensitive to the number of bins chosen and bin width

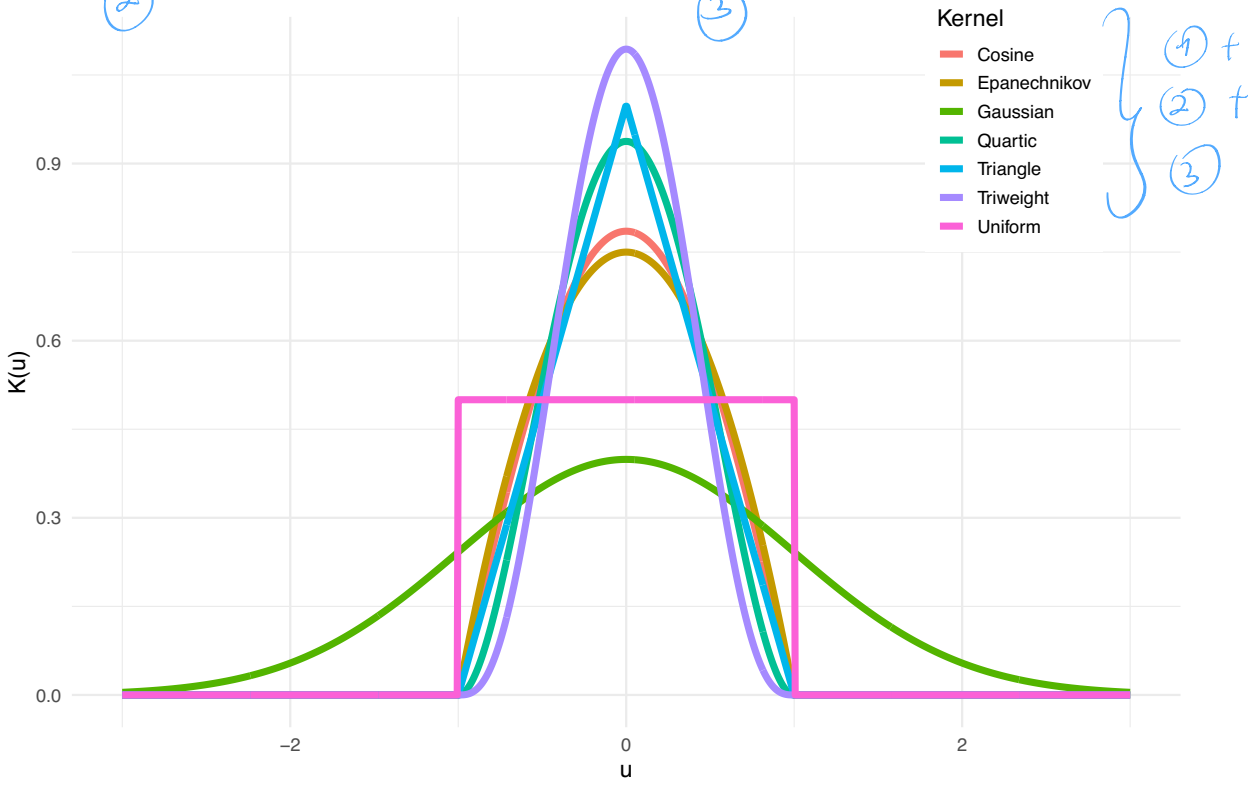


Kernel density estimation

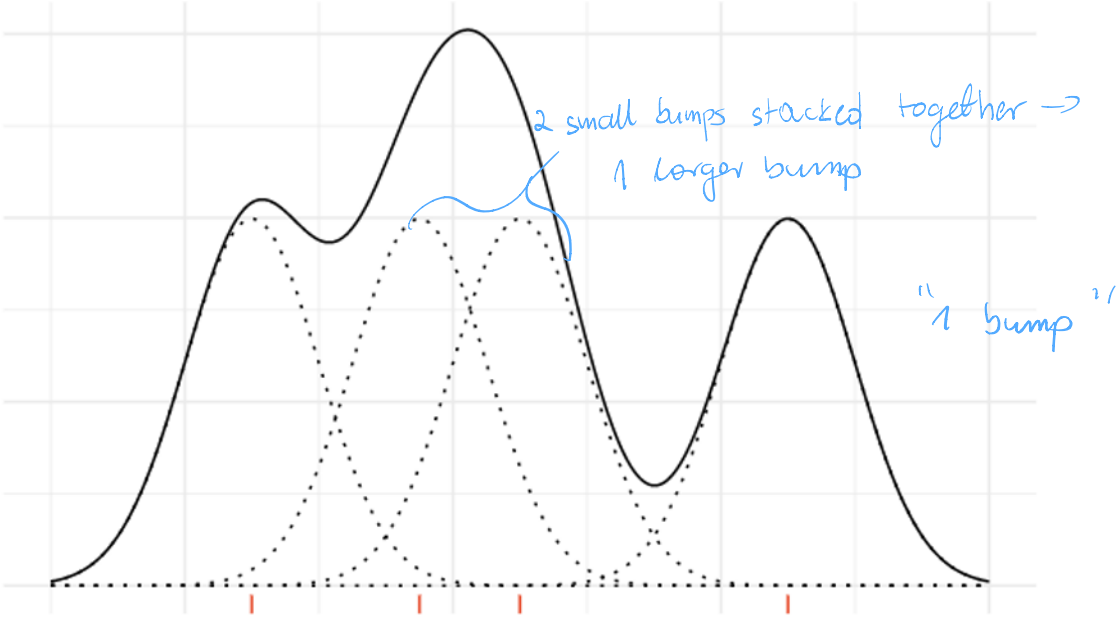
- Kernel density estimation is a nonparametric approach to density estimation.
- At each data point, a kernel function is centered at that observation.
- The density is estimated by summing these kernel functions and dividing by the number of observations, so that the estimator satisfies:
  - non-negativity,
  - the integral over its support equals 1.

Kernel functions

A kernel $K(\mathbf{x})$ is a density function with following properties: a) non-negative $K(\mathbf{x}) \geq 0$, b) symmetric $K(-\mathbf{x}) = K(\mathbf{x})$, c) unit measure $\int K(\mathbf{x})d\mathbf{x} = 1$.



Normal kernel density estimate



Example: Four sampled variables marked in red with Gaussian weights sum together to give the overall density estimate.

Kernel density estimator (KDE)

- A simple estimator weights all points within a window of width $2h$ around x equally:

$$\hat{f}(x) = \frac{1}{2nh} \sum_{i=1}^{n} 1_{\{|X_i - x| \leq h\}}$$

- More generally, a univariate kernel density estimator uses a kernel function K to assign weights:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - X_i}{h}\right)$$

where:

- K is a kernel function
- h is a bandwidth parameter, which may be fixed or vary with x

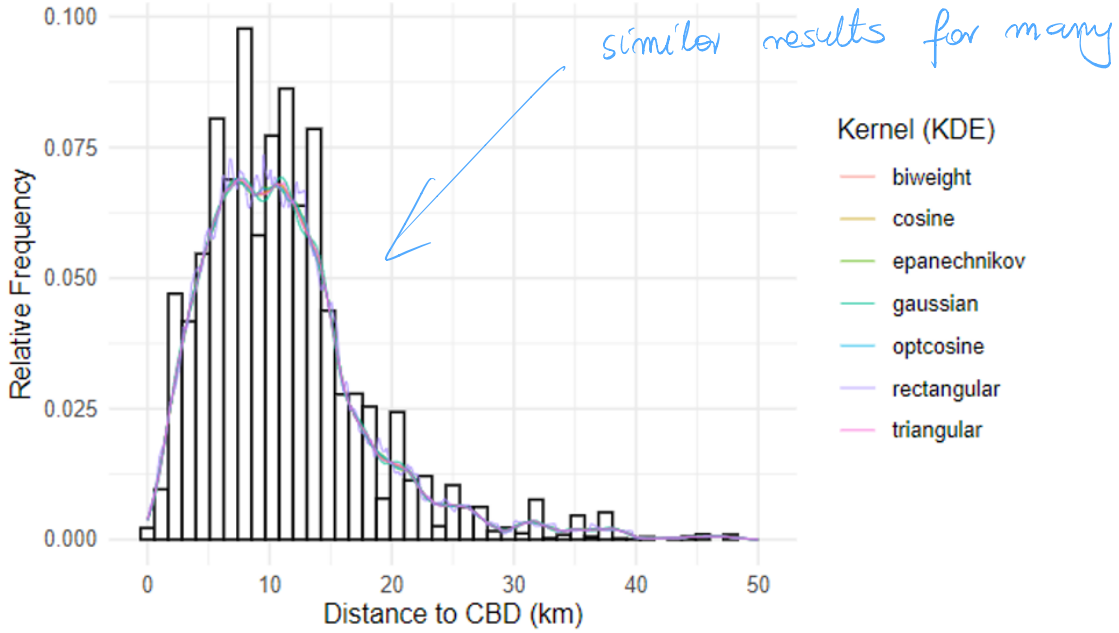
Tuning the kernel density estimator (KDE)

- The choice of kernel K is usually less important, as different kernels often produce similar results.
- The choice of bandwidth h is crucial, as it controls the degree of smoothing.

Some widely used kernels:

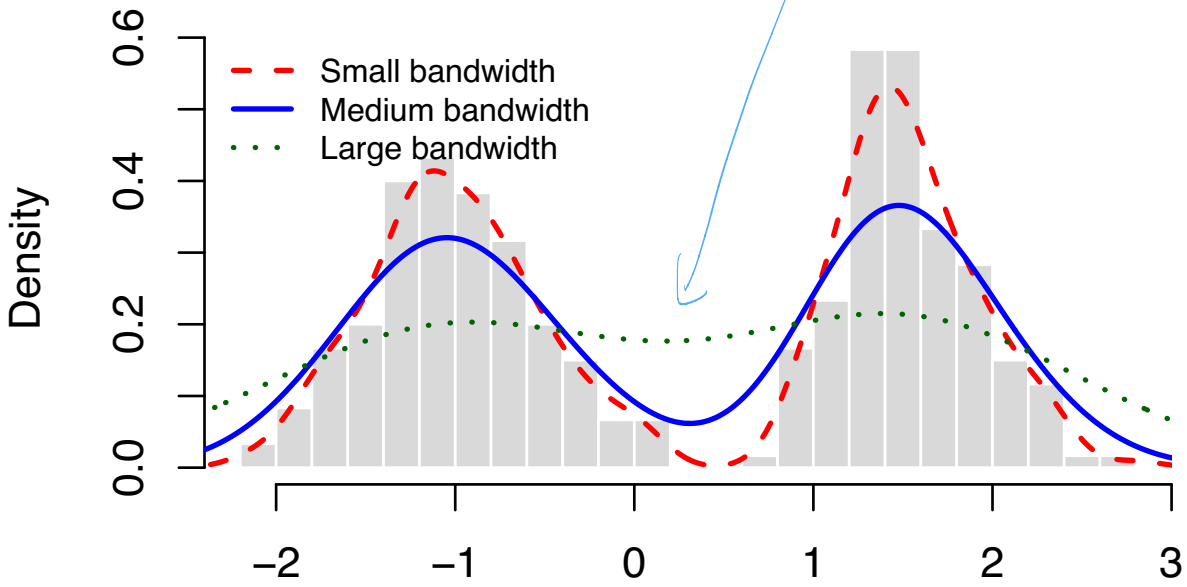
- Uniform: $K(x) = \frac{1}{2} 1_{\{|x| \leq 1\}}$
- Gaussian: $K(x) = \frac{1}{\sqrt{2\pi}} \exp\left\{-\frac{x^2}{2}\right\}$
- Epanechnikov: $K(x) = \frac{3}{4}(1 - x^2) 1_{\{|x| \leq 1\}}$

Different choices of kernel function with the same bandwidth



Choice of bandwidth

- If h is too small, the density estimator assigns probability density too locally near the observed data $\rightarrow$ *a wiggly estimated density with many spurious modes.*
- If h is too large, the density estimator spreads probability density contributions too diffusely $\rightarrow$ important features of f may be smoothed away,



Bandwidth selection by leave-one-out (LOO) cross-validation

The bandwidth h controls the smoothness of the KDE. A common criterion is based on the integrated squared error:

$$R(h) = \int \{\hat{p}_h(x) - p(x)\}^2 \, dx.$$

Since the true density p is unknown, we minimize an empirical version of this risk. Ignoring the constant term $\int p^2(x)dx$, the leave-one-out cross-validation criterion is

$$\widehat{R}_{\mathrm{LOO}}(h) = \int \hat{p}_h^2(x)dx - \frac{2}{n}\sum_{i=1}^{n} \hat{p}_{h,-i}(X_i),$$

where $\hat{p}_{h,-i}$ is the KDE computed after removing observation X_i.

$$\hat{h}_{\mathrm{LOO}} = \arg\min_{h \in \mathcal{H}} \widehat{R}_{\mathrm{LOO}}(h).$$

Idea: evaluate how well the density estimated from all observations except X_i predicts the omitted observation X_i. This avoids evaluating the estimator on the same observation that helped construct it.

$$R(h) = \int \left(\hat{p}_h(x) - p(x) \right)^2 dx$$

$$R(h) = \underbrace{\int \hat{p}_h^2(x)\, dx}_{①} - 2\underbrace{\int \hat{p}_h(x)\, p(x)\, dx}_{②} + \underbrace{\int p^2(x)\, dx}_{\text{doesn't depend on } h}$$

↓ bandwidth selection → focus on ① + ②

① $\int \hat{p}_h^2(x)\, dx \Rightarrow$ can be computed directly, depends only on $\hat{p}_h^2(x)$

$$= \int \left[\frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h} \right) \right]^2 dx = \frac{1}{n^2 h^2} \sum_{i=1}^{n} \sum_{j=1}^{n} \int K\left(\frac{x - x_i}{h} \right) K\left(\frac{x - x_j}{h} \right) dx$$

finite sums → swapping $\int$ with $\sum$ ok by lin.

$$z = \frac{x - x_i}{h}$$

$$x = x_i + hz$$

$$dx = h\, dz$$

$$I_{ij} = h \int K(z)\, K\left(\frac{x_i + hz - x_j}{h} \right) dz = h \int K(z)\, K\left(z + \frac{x_i - x_j}{h} \right) dz$$

convolution

$$v = \frac{x_i - x_j}{h} \Rightarrow K^{(2)}(v) = \int K(z)\, K(z + v)\, dz \Rightarrow I_{ij} = h\, K^{(2)}\left(\frac{x_i - x_j}{h} \right)$$

$$\Rightarrow \int \hat{p}_h^2(x)\,dx = \frac{1}{n^2 h^2} \sum_{i=1}^{n} \sum_{j=1}^{n} h\, K^{(2)}\left(\frac{x_i - x_j}{h}\right)$$

and replace with a specific kernel

(2) $$\int \hat{p}_h(x)\, p(x)\,dx = E_p[\hat{p}_h(x)] \approx \frac{1}{n} \sum_{i=1}^{n} \hat{p}_h(x_i)$$

using $\hat{p}_h(x_i)$ too optimistic →
x_i used to construct $\hat{p}_h(x_i)$

↳ use $\hat{p}_{h,-i}(x)$ instead

$$\approx \frac{1}{n} \sum_{i=1}^{n} \hat{p}_{h,-i}(x_i)$$

(final formula follows)

Bandwidth selection by data splitting (2-fold CV)

Split the data into a training sample and a validation sample:

$$X = (X_1, \ldots, X_{2n}) \quad \Rightarrow \quad \begin{cases} Y = (Y_1, \ldots, Y_n) & \text{training sample} \\ Z = (Z_1, \ldots, Z_n) & \text{validation sample} \end{cases}$$

For each candidate bandwidth $h_j \in \mathcal{H}$, estimate a density using only the training data: $Y \longrightarrow \hat{p}_j = \hat{p}_{h_j}$. Then evaluate its estimated risk on the validation sample:

$$\widehat{L}_j = \int \hat{p}_j^2(x)dx - \frac{2}{n}\sum_{i=1}^{n} \hat{p}_j(Z_i).$$

replaced by the density computed on the validation set

Choose the density estimator with the smallest estimated risk:

$$\hat{p} = \hat{p}_{\hat{j}}, \qquad \hat{j} = \arg\min_{j=1,\ldots,N} \widehat{L}_j.$$

Idea: use one part of the data to construct candidate KDEs, and an independent part of the data to compare their predictive performance.

Exemplar and kernel methods

- Exemplar methods
  - k-nearest neighbor (kNN)
  - kernel density estimation (KDE)
- Kernel methods
  - **Mercer Kernels**
  - Gaussian process (GP)
  - Support Vector Machine (SVM)

Kernel methods: main idea

Many learning algorithms depend on the data only through inner products $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$.

Given training inputs $x_1, \ldots, x_N$, the matrix of all pairwise inner products is called the **Gram matrix**: $G_{ij} = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$, $i, j = 1, \ldots, N$.

Kernel methods replace these inner products by a kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$, giving the **kernel Gram matrix** $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$.

If $K(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle_{\mathcal{H}}$, then the kernel Gram matrix is the Gram matrix of the transformed features $\phi(\mathbf{x}_1), \ldots, \phi(\mathbf{x}_N)$.

Here $\phi(\mathbf{x})$ maps the original input $\mathbf{x}$ into a possibly high-dimensional feature space $\mathcal{H}$.

The advantage is that we can work with $K(\mathbf{x}, \mathbf{z})$ directly, without explicitly computing $\phi(\mathbf{x})$ and $\phi(\mathbf{z})$. This is the **kernel trick**.

$$\hat{\beta}_{PLS} = (X^T X + \lambda I)^{-1} X^T y \qquad \hat{f}(x) = x^T \hat{\beta}$$

$$(X^T X + \lambda I)^{-1} X^T = X^T (X X^T + \lambda I)^{-1}$$

So

$$\hat{\beta}_{PLS} = X^T \overbrace{(X X^T + \lambda I)^{-1} y}^{\alpha} = X^T \alpha$$

$$= \sum_{i=1}^{n} \alpha_i x_i$$

$$\hat{f}(x) = x^T \hat{\beta} = x^T \sum_{i=1}^{n} \alpha_i x_i \overset{\text{linearity}}{=} \sum_{i=1}^{n} \alpha_i x_i^T x$$

$$= \sum_{i=1}^{n} \alpha_i \langle x_i^T, x \rangle$$

Mercer kernels

A function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a valid kernel if it behaves like an inner product in some feature space.

Standard sufficient conditions:

- K is symmetric: $K(\mathbf{x}, \mathbf{x}') = K(\mathbf{x}', \mathbf{x})$
- K is positive semidefinite: for any $\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathcal{X}$ and any $c_1, \ldots, c_N \in \mathbb{R}$

$$\sum_{i=1}^{N} \sum_{j=1}^{N} c_i c_j K(x_i, x_j) \geq 0.$$

Equivalently, the Gram matrix $\mathbf{K}_{ij} = K(x_i, x_j)$ must be positive semidefinite:

$$c^\top \mathbf{K} c \geq 0 \qquad \text{for all } c \in \mathbb{R}^N.$$

Example: polynomial kernel

implicit transformation of features

Consider the degree-2 polynomial kernel $K(\mathbf{x}, \mathbf{z}) = \langle \mathbf{x}, \mathbf{z} \rangle^2$.

For $\mathbf{x} = (x_1, x_2)^\top$ and $\mathbf{z} = (z_1, z_2)^\top$, $K(\mathbf{x}, \mathbf{z}) = (x_1 z_1 + x_2 z_2)^2$.

Expanding, $K(\mathbf{x}, \mathbf{z}) = x_1^2 z_1^2 + 2 x_1 x_2 z_1 z_2 + x_2^2 z_2^2$.

This is an inner product in the transformed feature space

$$\phi(\mathbf{x}) = \left(x_1^2, \sqrt{2} x_1 x_2, x_2^2\right)^\top.$$

2 D → 3 D (finite # of features)

not unique

Indeed, $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$. ← inner product as if the data were transformed

For example, a linear model in the transformed variables θ

$$f(\mathbf{x}) = \theta^\top \phi(\mathbf{x}) = \theta_1 x_1^2 + \theta_2 \sqrt{2} x_1 x_2 + \theta_3 x_2^2.$$

is nonlinear in the original variables x_1, x_2.

Kernel methods use $K(\mathbf{x}, \mathbf{z})$ to work with these transformed features without explicitly constructing $\phi(\mathbf{x})$.

Example: Gaussian / RBF kernel

radial basis fct.

Gaussian or radial basis function (RBF) kernel is common:

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\ell^2}\right).$$

The parameter $\ell > 0$ is the length-scale, or bandwidth. (h in KDE)

- Small ℓ: only very nearby points have high similarity.
- Large ℓ: points remain similar over longer distances.

The RBF kernel is positive semidefinite $\rightarrow$ a valid Mercer kernel.

It corresponds to an implicit feature map $\phi(\mathbf{x})$ into an infinite-dimensional feature space: $K(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle_{\mathcal{H}}$.

Thus kernel methods can fit nonlinear functions while avoiding explicit construction of high-dimensional features.

$$K(x,z) = \exp\left(-\frac{\|x-z\|^2}{2\ell^2}\right)$$

$p = 1$

$(x-z)^2 = x^2 - 2xz + z^2$ cross-product terms

$$K(x,z) = \exp\left(\frac{-x^2}{2\ell^2}\right)\exp\left(\frac{xz}{\ell^2}\right)\exp\left(\frac{-z^2}{2\ell^2}\right)$$

$$e^t = 1 + t + \frac{t^2}{2} + \frac{t^3}{3!} + \ldots \qquad \frac{xz}{\ell^2} = t$$

kernel contains terms involving $1, x, x^2, x^3, x^4$

$$\phi(x) = \exp\left(\frac{-x^2}{2\ell^2}\right)\left(1, \frac{x}{\ell}, \frac{x^2}{\sqrt{2!}\,\ell^2}, \frac{x^3}{\sqrt{3!}\,\ell^3}, \ldots\right)$$

$$K(x,z) = \langle \phi(x), \phi(z) \rangle$$

$x \in \mathbb{R}^d \rightarrow$ monomials $x_1x_2, x_1^2, x_2^2, x_1x_2, \ldots x_1^2x_2 \ldots$

Exemplar and kernel methods

- Exemplar methods
  - k-nearest neighbor (kNN)
  - kernel density estimation (KDE)
- Kernel methods
  - Mercer Kernels
  - **Gaussian process (GP)**
  - Support Vector Machine (SVM)

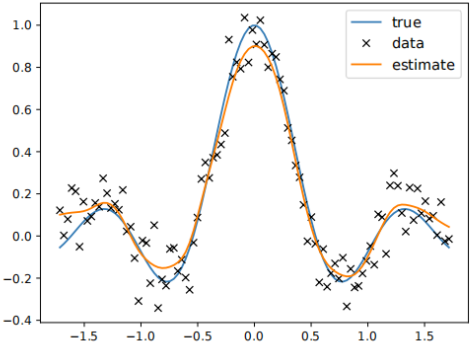
Warm-up: Nonparametric Regression

We observe $(X_1, Y_1), \ldots, (X_n, Y_n)$ and we want to estimate $m(\mathbf{x}) = E(Y|\mathbf{X} = \mathbf{x})$

Idea

Use the **Nadaraya–Watson estimator** with a smoothing kernel K and a bandwidth $h > 0$ controlling the amount of smoothing (nearby observations receive larger weights).

$$\hat{m}_h(\mathbf{x}) = \frac{\sum_{i=1}^n Y_i \, K\left(\frac{\|\mathbf{x}-\mathbf{X}_i\|}{h}\right)}{\sum_{i=1}^n K\left(\frac{\|\mathbf{x}-\mathbf{X}_i\|}{h}\right)}.$$



$\bar{y} = \frac{\sum_{i=1}^n Y_i w_i}{\sum w_i} = \sum_{i=1}^n Y_i w_i'$

$w_i' = \frac{K\left(\frac{x-x_i}{h}\right)}{\sum_{i=1}^n K\left(\frac{x-x_i}{h}\right)}$

(weighted arithmetic mean with more sophisticated weights)

Gaussian Processes: High-Level Idea

A **Gaussian process (GP)** is a distribution over functions $f : \mathcal{X} \to \mathbb{R}$

We write $f \sim \mathcal{GP}(m, \mathcal{K})$, where

- $m(\mathbf{x})$ is the **mean function**,
- $\mathcal{K}(\mathbf{x}, \mathbf{x}')$ is the **kernel** or **covariance function**.

The GP assumption means that for any finite collection of inputs $\mathbf{x}_1, \ldots, \mathbf{x}_n$, the vector of function values is multivariate Gaussian:

$$\begin{pmatrix} f(\mathbf{x}_1) \\ \vdots \\ f(\mathbf{x}_n) \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} m(\mathbf{x}_1) \\ \vdots \\ m(\mathbf{x}_n) \end{pmatrix}, K_{X,X} \right),$$

Kernel -> similarity measure

where $(K_{X,X})_{ij} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$.

The kernel controls how strongly function values at different inputs are related.

Kernels in Nadaraya–Watson vs. in Gaussian processes

- Nadaraya–Watson regression uses a kernel to define **local similarity**.
- Gaussian processes uses a kernel $K(\mathbf{x}, \mathbf{z})$ to define the **covariance** between function values $f(\mathbf{x})$ and $f(\mathbf{z})$.
- **The kernel moves from being a local averaging weight to defining a full probabilistic model over functions**.

local averaging → parameter defining a probabilistic model

Noise-Free GP Regression

In the noise-free case, the observations are assumed to lie exactly on the unknown function: $y_i = f(\mathbf{x}_i)$.

This means that if we observe $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)$, then the GP must pass through these observed points.

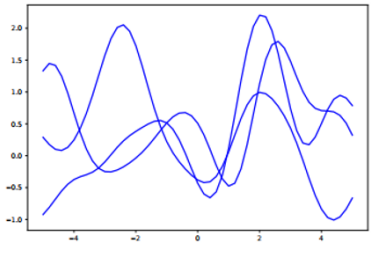
The model treats the observed outputs as exact function values. Predictions at new points are made by conditioning on the observed data: $p(f_* \mid \mathbf{x}_*, \mathcal{D})$.

The resulting posterior mean interpolates the data, while the posterior uncertainty is small near observed points and larger farther away.

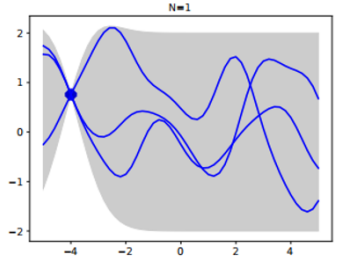
GP without noise – illustration

Functions sampled from a GP prior

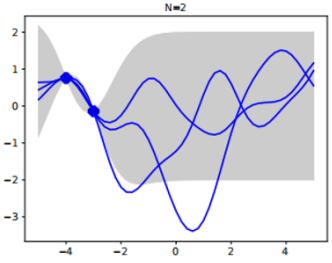
GP posterior after conditioning on 1 noise-free obs



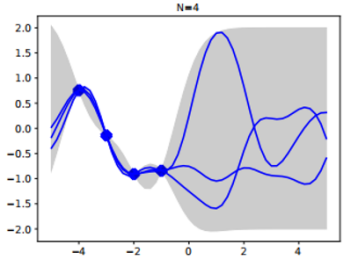
(a)



(b)



(c)



(d)

— 11 — 2 noise-free obs

4 — obs

Noisy Relationships

In many applications, observations are not exact. Instead, we observe the function with measurement noise: $y_i = f(\mathbf{x}_i) + \epsilon_i$, where typically $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$.

Now the GP does **not** need to pass exactly through every observed point. The data are treated as noisy measurements of an underlying smooth function.

The noise term adds extra uncertainty to the observed outputs:

$$\text{Cov}(\mathbf{y}) = K_{X,X} + \sigma^2 I.$$

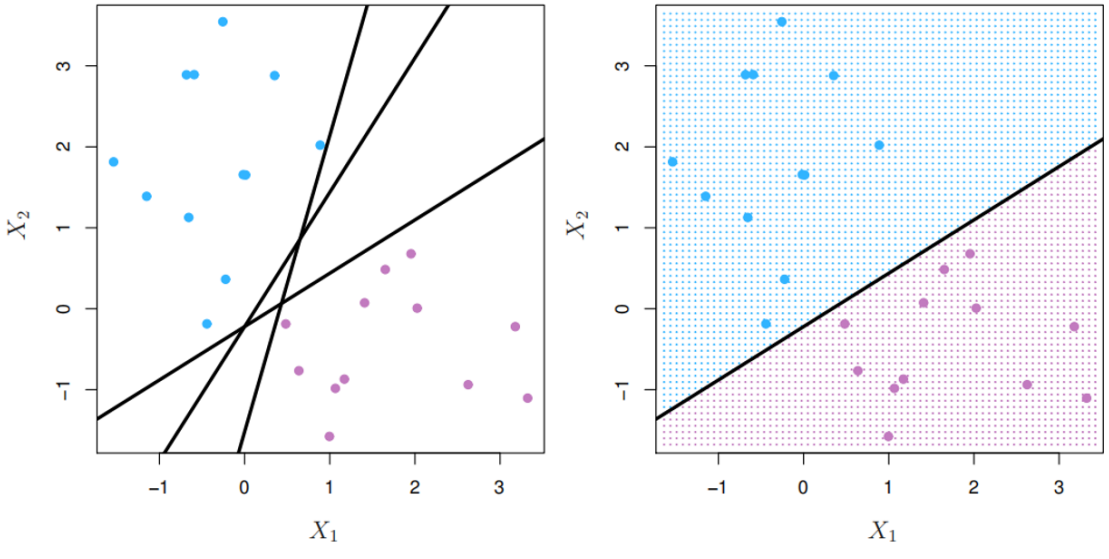
This makes the GP behave like a smooth regression model: it follows the general pattern in the data, but does not overreact to individual noisy observations.

Exemplar and kernel methods

- Exemplar methods
  - k-nearest neighbor (kNN)
  - kernel density estimation (KDE)
- Kernel methods
  - Mercer Kernels
  - Gaussian process (GP)
  - Support Vector Machine (SVM)

What is a "good" classifier?

Or, which classifier is better?



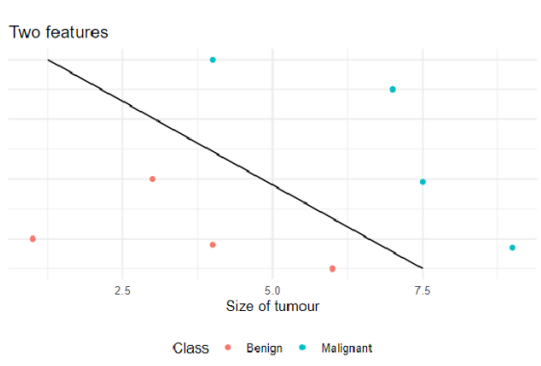
Hyperplane → useful in classification

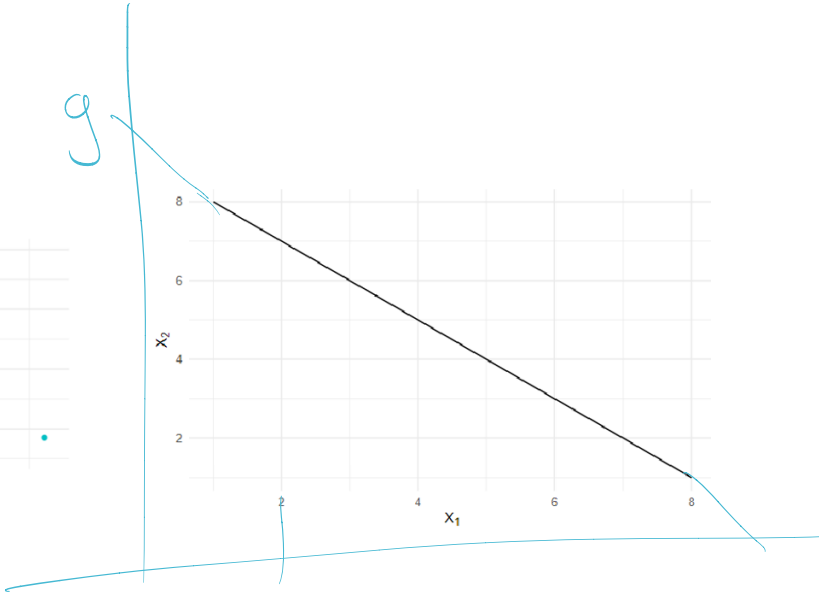
- In p dimensions, it is a flat affine subspace of dimension $p - 1$
- General equation has the form: $\Rightarrow b^T x + \beta_0 = 0$

$$\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p = 0$$

- In $p = 2$ dimensions, the hyperplane is a line $\mathbb{R}^3 \rightarrow$ plane
- If $\beta_0 = 0$, the hyperplane passes through the origin, otherwise it does not.
- The vector $(\beta_0 \ldots, \beta_p)$ is called the normal vector → it points to a direction orthogonal to the surface of the hyperplane.

Hyperplane in $p = 2$





Equation of the hyperplane here is $-9 + X_1 + X_2 = 0$

$X_2 = -X_1 + 9$

Separating hyperplanes

Consider class 1 (red) as $Y_i = 1$ and class 2 (blue) as $Y_i = -1$.

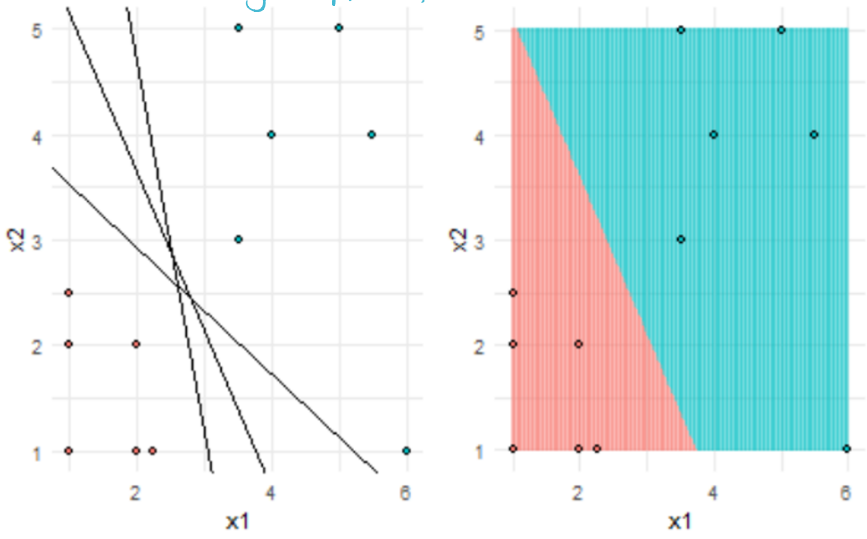
We want $f(x_i)$ to have the same sign as y_i $\Rightarrow$

Then $y_i f(x_i) > 0$ for all i; $f(x_i)$ defines a separating hyperplane.

If $f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$ defines a hyperplane:

- $f(x) > 0$ defines a region on one side of the hyperplane,
- $f(x) < 0$ on the other side.

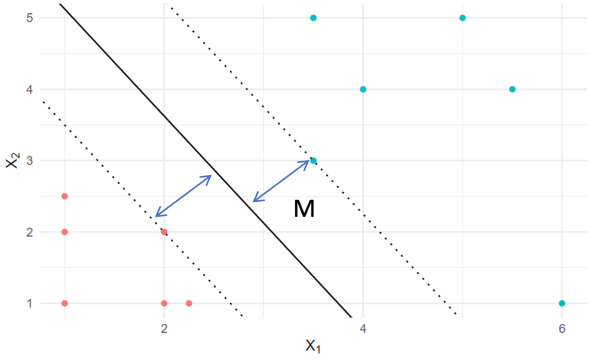
many diff options



Maximal Margin Classifier

→ Find a separating line that
- separates blue from red
- does so with a largest possible gap

M



$$\max_{\beta_0,\beta_1,\beta_2,\ldots,\beta_p} M \quad \text{such that} \quad \sum_{j=1}^{p} \beta_j^2 = 1$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) \geq M$$

f(x) before 0

- Given a hyperplane, what is the distance between a point to the hyperplane?
- What is the "optimal" way to seperate samples from two classes?
- Which part of samples are more "important" to determine the hyperplane?

max margin

2D: $1 - x_1 - x_2 = 0$

$\Leftrightarrow 10 - 10x_1 - 10x_2 = 0$

can be written in many ways, to prevent rescaling

2-D case

1. $\max M$ s.t. $\|\beta\| = 1$ $\quad y_i(\beta_0 + x_i\beta) \geq M$

(max the dist from any point to the sep. plane

2. Drop the $\|\beta\| = 1$ +

Margin => perpendicular dist. from x_i to the hyperplane is given by

$$\frac{y_i(\beta_0 + x_i\beta)}{\|\beta\|}$$

3. Choose the scale c such that (instead of diff #)

$$\min_i y_i(\beta_0 + x_i\beta) = 1$$

$$\Rightarrow \text{all } y_i(\beta_0 + x_i\beta) \geq 1$$

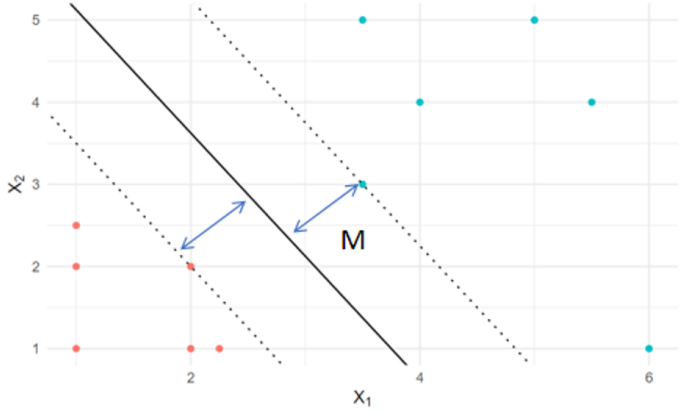
4. Max M $\Leftrightarrow$ min $\|\beta\|$

Maximal Margin Classifier

After some re-formulation and reparameterization, we have

$$min_{\beta_0,\beta_1,\ldots\beta_p} \frac{1}{2}||\beta||^2$$

$$\text{such that } y_i(x_i\beta + \beta_0) \geq 1$$



quadratic programming!

From the primal to the dual

The hard-margin SVM starts with the primal problem:

$$min_{\beta_0,\beta_1,\ldots\beta_p} \frac{1}{2}||\beta||^2$$
$$\text{such that } y_i(x_i\beta + \beta_0) \geq 1$$

Interpretation:

- We want a separating hyperplane.
- We want the margin to be as large as possible.
- Every training point must be on the correct side of the margin.

The dual version rewrites the problem using one weight per training point: $\alpha_i \geq 0$.

Think of α_i as the **importance weight** of observation i.

The key Karush-Kuhn-Tucker (KKT) idea

left without any proof

The important KKT condition is: $\alpha_i \left(y_i f(x_i) - 1\right) = 0$. This means one of the two terms must be zero.

Case 1: Point is far from the margin: $y_i f(x_i) > 1$

Then $y_i f(x_i) - 1 > 0$ so we must have: $\alpha_i = 0$.
This point does **not** affect the final classifier.

Case 2: Point is exactly on the margin: $y_i f(x_i) = 1$

Then α_i may be positive.
These points are the **support vectors**.

Support vectors

The final SVM classifier can be written as:

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i x_i^\top x.$$

But most α_i's are zero. So only points with $\alpha_i > 0$ actually contribute to the classifier.

These points are called **support vectors**.

Main takeaway:

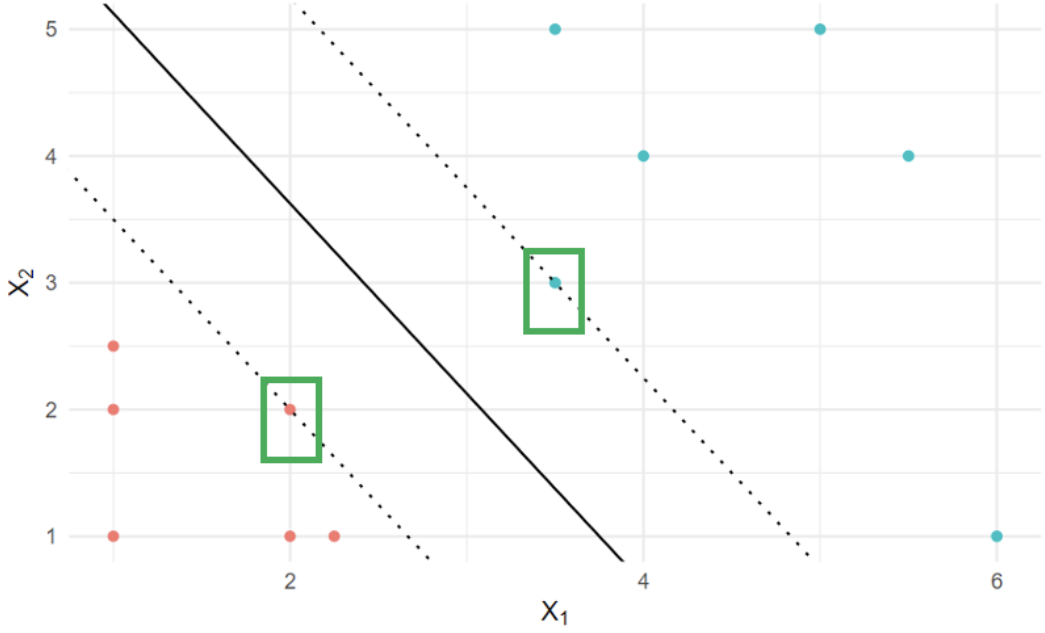
The SVM boundary is determined only by the support vectors.

Points far away from the margin do not matter, because their $\alpha_i = 0$.

Illustration

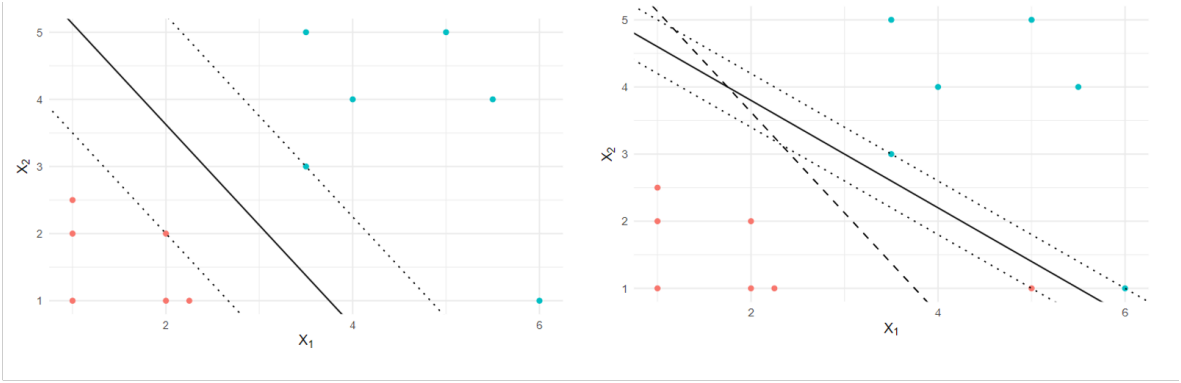
Only $\alpha_i > 0$ contribute to final classification function

Only $(y_i f(x_i) - 1) = 0$ could lead to $\alpha_i > 0$



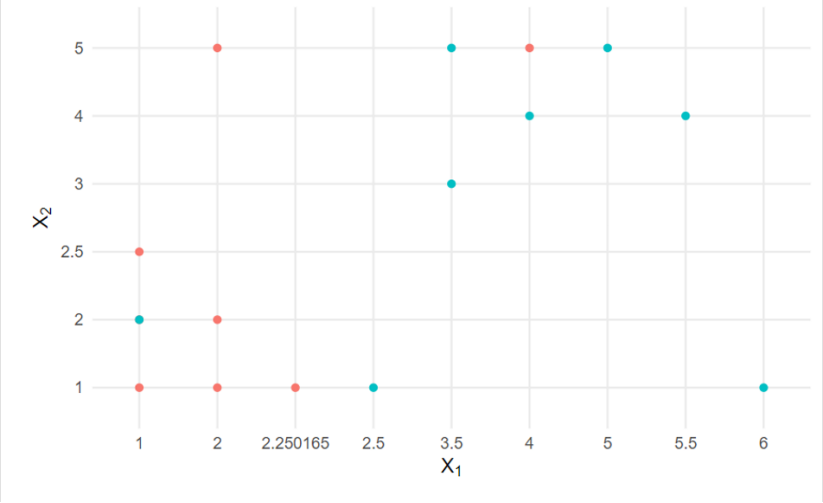
Support vector/support set

Effect of noisy data



- Data could be separable, but noisy $\rightarrow$ unstable solution for the maximal margin classifier.
- The support vector classifier maximizes a soft margin.
  - relaxes requirement for all observations to be on the correct side of the margin.

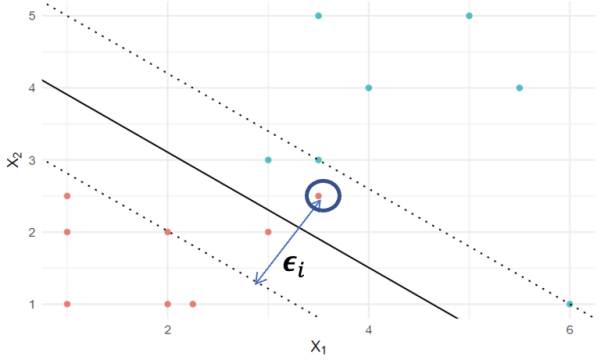
Non-perfect separation



- There is no linear boundary (hyperplane) that perfectly separates the classes

- This is typically the case that observations don't have a perfect boundary of separation
  - relaxes requirement for **all observations** to be on the correct side of the margin.

Slack variables in support vector classifier



measures how much point i is allowed to violate M

as before

$$\max_{\beta_0,\beta_1,\beta_2,\ldots,\beta_p,\ \epsilon_1,\ldots,\epsilon_n} M \quad \text{such that} \quad \sum_{j=1}^{p} \beta_j^2 = 1$$

$$y_i\left(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}\right) \geq M(1-\epsilon_i)$$

each obs gets a slack var

$$\epsilon_i \geq 0, \qquad \sum_{i=1}^{n} \epsilon_i \leq C$$

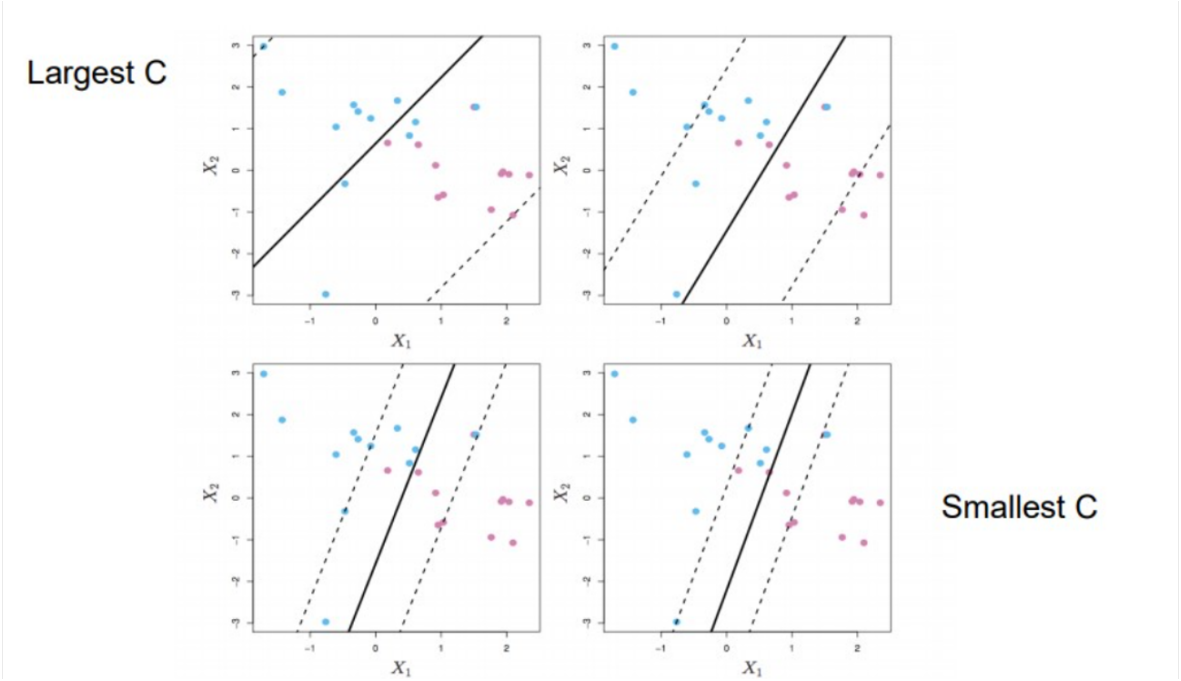
- $\epsilon_i = 0$ → hard
- $0 < \epsilon_i < 1 \Rightarrow 0 < M(1-\epsilon_i) < M$ point classified OK but inside the margin
- $\epsilon_i = 1$ ⇒ point on bound ; $\epsilon_i > 1$ ⇒ point on the wrong side

slack weakens hard-margin version

Impact of parameters C

Small C : little violation allowed

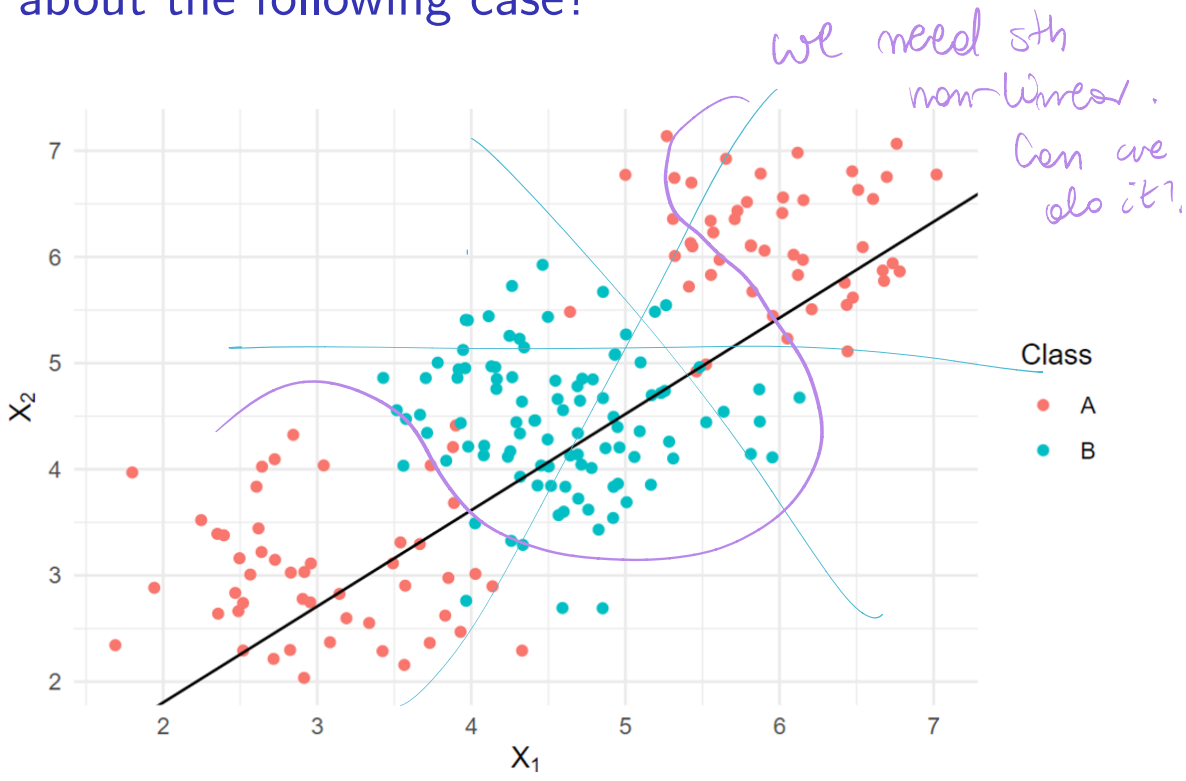
large C : more violation allowed



Quick summary

- Hyperplane
- Maximal margin classifier
- Support vector classifier

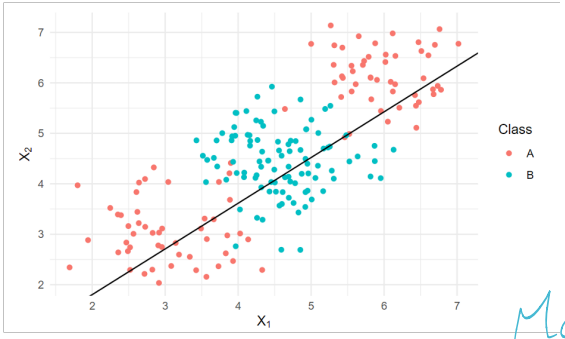
How about the following case?



- Single linear boundary can be insufficient

Feature space expalsion

- For the data points not (linear) separable in lower dimensional space, if we map (denote as ϕ) them into higher dimensional space, it is more likely to be separable. (VC dimension!)



Map to a h-dim space (from 2D to 3D) and classify

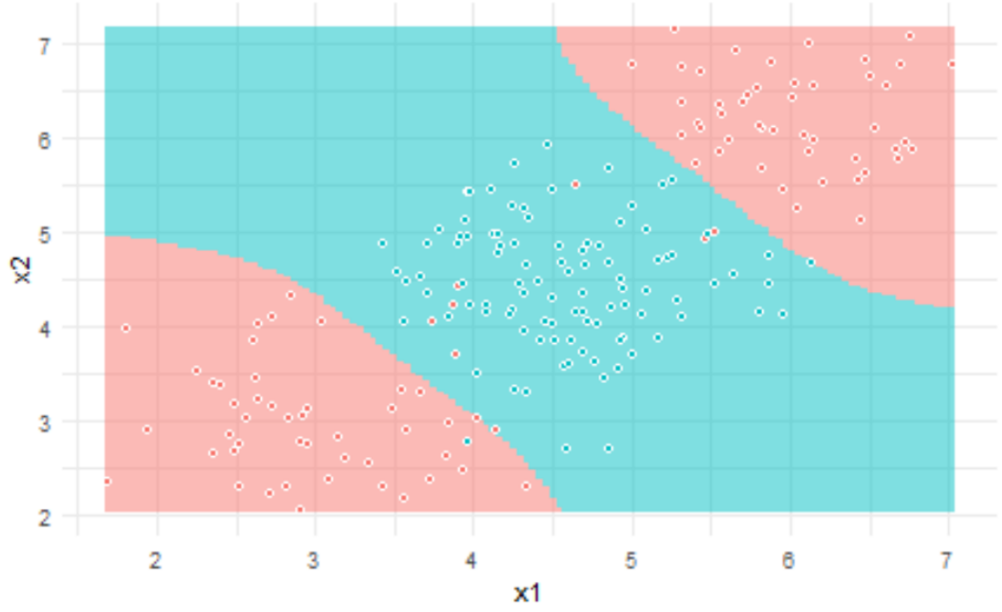
$$\phi : (X_1, X_2) \rightarrow (X_1, X_2, X_1^2, X_2^2, X_1X_2)$$

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$$

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 = 0$$

Feature space expansion

- Using degree six polynomial expansion



- In general, after mapping, decision boundary can be written as:

$$\{x \in \mathbb{R}^p : \beta_0 + \phi(x)\beta = 0\}$$

Kernel tricks

- Recall inner product:
- Let $x_i = (x_{i1}, \ldots, x_{ip})$, the inner product between vectors given by $< x_i, x_j > = \sum_{k=1}^{p} x_{ik} x_{jk}$
- The linear support vector classifier can be represented as (duality)

$$f(x) = \beta_0 + \sum_{i=1}^{N} \alpha_j \langle x, x_i \rangle$$

replace by a kernel

$$f(x) = \beta_0 + \sum_{i=1}^{N} \alpha_j \underbrace{\langle \phi(x), \phi(x_i) \rangle}_{K(x, x_j)}$$

Support vector machine

- The support vector machine (SVM) is an extension of the support vector classifier that obtains non-linear decision boundaries by (implicitly) enlarging the feature space using kernels.

$$f(x) = \beta_0 + \sum_{i=1}^{N} \alpha_i K(x, x_i)$$

- As before, the classifier will be based on a hyperplane (in the higher dimensional space). The classifier will have the form:

$$\hat{y} = sign(f(x))$$

$y \cdot f(x) > 0$
< 0

Some kernels

- Polynomial kernel

$$K(x_i, x_j) = (1 + \langle x_i, x_j \rangle)^d$$

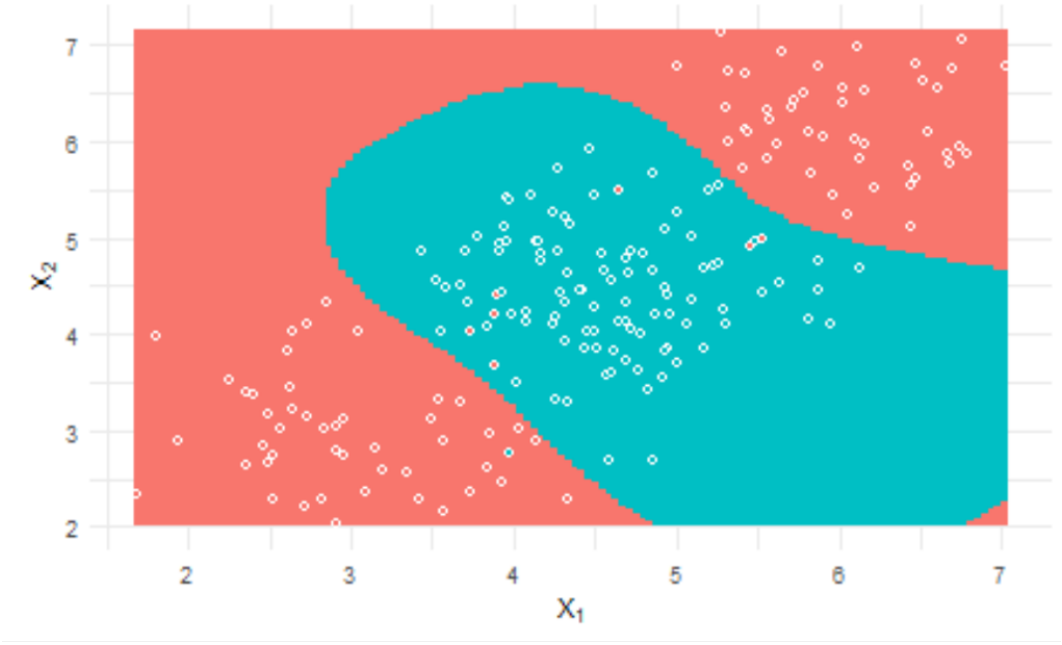
- Gaussian radial kernel/Radial basis function kernel

$$K(x_i, x_j) = exp(-\gamma ||x_i - x_j||^2)$$

- Gaussian radial kernel (1d) as expansion of feature mapping:

$$\phi(x) = e^{-\gamma x^2} \left[1, \sqrt{\frac{\gamma}{1!}}\, x, \sqrt{\frac{\gamma^2}{2!}}\, x^2, \ \ldots, \ \sqrt{\frac{\gamma^n}{n!}}\, x^n, \ \ldots \right]$$

Support vector machines with the radial kernel



SVM for multiclass

- SVM covered previously are design for binary classification. To expand this to K classes there are two options:

1. One vs one:
   Fit all $\binom{K}{2}$ pairwise classifiers. Then, classify x to the class that wins the most pairwise competitions.
2. One vs all:
   Fit k different 2-class SVM classifiers; each class versus the rest. Classify x to the class for which the signed distance to the decision boundary is the largest.

- Which to use?

If k is small, do one vs one. Otherwise recommended one vs all.

References

- ESLII: Chapters 3.1, 4.1-4.4
- PRML Chapters 3.1, 4.1
- PML: Chapter 16-17